

F24-029-D-VerseCraft

Project Team

Saad Ahmed Qureshi	21I-0616
Syed Murtaza Kazmi	21I-0685
Muhammad Waleed Yaseen	21I-0438

Session 2021-2025

Supervised by

Mr. Owais Idrees

Co-Supervised by

Dr. Faisal Cheema



Department of Computer Science

**National University of Computer and Emerging Sciences
Islamabad, Pakistan**

June, 2025

Contents

1	Introduction	1
1.1	Problem Statement	1
1.2	Scope	1
1.3	Modules	2
1.3.1	Module 1: Workspace Dashboard	2
1.3.2	Module 2: AI-Analysis Tools	2
1.4	User Classes and Characteristics	3
2	Project Requirements	5
2.1	Use-case/Event Response Table/Storyboarding	5
2.2	Detailed Use Cases	7
2.2.1	Use Case 1: User Registration and Login	7
2.2.2	Use Case 2: Create and Manage Writing Projects	9
2.2.3	Use Case 3: Manage Chapters	10
2.2.4	Use Case 4: Manage Notes (Inline Notes)	12
2.2.5	Use Case 5: Collaborative Writing	14
2.2.6	Use Case 6: View and Comment on Published Projects	16
2.2.7	Use Case 7: Manage Account Settings	17
2.2.8	Use Case 8: AI-Assisted Grammar Check	19
2.2.9	Use Case 9: Manage Characters	21
2.2.10	Use Case 10: Switch Language (English/Urdu)	22
2.2.11	Use Case 11: Customize Workspace	24
2.2.12	Use Case 12: Export Project (PDF, DOCX, etc.)	26
2.2.13	Use Case 13: Version Control	27
2.2.14	Use Case 14: AI-Assisted Writing Assistant (ChatBot)	29
2.2.15	Use Case 15: Manage Project (Edit, Delete)	31
2.2.16	Use Case 16: View Analytics and Writing Metrics	33
2.2.17	Use Case 17: Publish Completed Project	35
2.2.18	Use Case 18: Manage Users and Projects (Admin)	37
2.3	Functional Requirements	38
2.3.1	Module 1: Workspace Dashboard	38
2.3.2	Module 2: AI-Analysis Tools	39

2.4	Non-Functional Requirements	39
2.4.1	Reliability	40
2.4.2	Usability	40
2.4.3	Performance	40
2.4.4	Security	40
2.5	Domain Model	40
3	System Overview	43
3.1	Architectural Design	43
3.1.1	Modular Program Structure	43
3.1.1.1	Architecture Layers	43
3.1.1.2	Module Interactions	45
3.1.1.3	High-Level System Diagram	45
3.1.1.4	Detailed Architecture Style: MVC (Model-View-Controller)	45
3.2	Design Models	46
3.2.1	Activity Diagram	46
3.2.2	Class Diagram	47
3.2.3	Sequence Diagrams	47
3.2.3.1	Use Case 1: User Registration and Login	47
3.2.3.2	Use Case 2: Create and View Writing Projects	48
3.2.3.3	Use Case 3: Manage Chapters	49
3.2.3.4	Use Case 4: Manage Notes	50
3.2.3.5	Use Case 5: Collaborative Writing	51
3.2.3.6	Use Case 6: View and Comment on Published Projects	52
3.2.3.7	Use Case 7: Manage Account Settings	53
3.2.3.8	Use Case 8: AI-Assisted Grammar Check	53
3.2.3.9	Use Case 9: Manage Characters	54
3.2.3.10	Use Case 10: Switch Language (English/Urdu)	54
3.2.3.11	Use Case 11: Customize Workspace	55
3.2.3.12	Use Case 12: Export Project (PDF, DOCX, etc.)	56
3.2.3.13	Use Case 13: Version Control	57
3.2.3.14	Use Case 14: AI-Assisted Writing Assistant (ChatBot)	58
3.2.3.15	Use Case 15: Manage Project (Edit, Delete)	59
3.2.3.16	Use Case 16: View Analytics and Writing Metrics	60
3.2.3.17	Use Case 17: Publish Completed Project	61
3.2.3.18	Use Case 18: Backup and Restore	62
3.3	Data Design	62
3.3.1	Data Transformation and Structures	62
3.3.2	Data Entities and Relationships	63
3.3.3	Database Schema	66

3.3.4	Storage and Processing	66
3.3.5	Data Flow	67
3.3.6	Security Measures	67
3.3.7	Scalability Considerations	67
3.3.8	Database Schema Diagram	68
3.3.9	Summary	68
4	Conclusions	69
	References	71

List of Figures

2.1	Use Case Diagram for VerseCraft	7
2.2	Domain Model of the System	41
3.1	High-Level System Diagram of VerseCraft	45
3.2	Activity Diagram for VerseCraft	46
3.3	Class Diagram for VerseCraft	47
3.4	Sequence Diagram for User Registration and Login	48
3.5	Sequence Diagram for Create and View Writing Projects	49
3.6	Sequence Diagram for Manage Chapters	50
3.7	Sequence Diagram for Manage Notes	51
3.8	Sequence Diagram for Collaborative Writing	52
3.9	Sequence Diagram for View and Comment on Published Projects	52
3.10	Sequence Diagram for Manage Account Settings	53
3.11	Sequence Diagram for AI-Assisted Grammar Check	54
3.12	Sequence Diagram for Manage Characters	54
3.13	Sequence Diagram for Switch Language (English/Urdu)	55
3.14	Sequence Diagram for Customize Workspace	56
3.15	Sequence Diagram for Export Project	57
3.16	Sequence Diagram for Version Control	58
3.17	Sequence Diagram for AI-Assisted Writing Assistant (ChatBot)	59
3.18	Sequence Diagram for Manage Project	60
3.19	Sequence Diagram for View Analytics and Writing Metrics	61
3.20	Sequence Diagram for Publish Completed Project	61
3.21	Sequence Diagram for Backup and Restore	62
3.22	Database Schema Diagram for VerseCraft	68

List of Tables

1.1 User Classes and Characteristics 3

Chapter 1

Introduction

The product specified in this document is VerseCraft, a web-based writing workflow management tool. The application aims to streamline the creative writing process and provide tools for collaboration, productivity, and AI-assisted content creation. This document is intended for a variety of readers, including developers who will build and maintain the application, project managers overseeing the project, marketing staff responsible for promotion, users of the platform, testers validating the system, and documentation writers. [1] [2] [3] .

1.1 Problem Statement

Writers often face challenges such as creative blocks, disorganized workflows, and limited grammar proficiency, leading to decreased productivity. Without tools to guide them when they lose direction, writers may struggle with plot or character development, leading to incomplete or unsatisfactory works. Additionally, many writers lack sufficient grammatical skills, especially when working in a second language like English or Urdu. Our software is being developed to solve these issues by providing a collaborative, structured writing environment, which integrates AI to assist with grammar checks and offer suggestions for creativity. Through a centralized platform, writers can work more efficiently, collaborate with others, and track their progress with tools designed to address these common hurdles.

1.2 Scope

The scope of VerseCraft encompasses the development of an AI-assisted writing platform that offers structured writing tools, collaboration features, and workflow management ca-

pabilities. The main functionalities include chapter management, modifiable outline bars, notes creation, version control, and options to write in both English and Urdu. The application will provide features like character and plot development windows, AI-powered grammar checking, and writing suggestions. It will also support publishing options with privacy controls. The boundaries of this project include focusing solely on providing an end-to-end solution for writers, without delving into other unrelated creative activities such as graphic design or video production. The solution will cater to both novice and experienced writers who require robust tools to manage their writing projects efficiently.

1.3 Modules

The proposed project consists of several modules designed to provide key functionalities to the users. Each module is outlined below:

1.3.1 Module 1: Workspace Dashboard

The workspace dashboard provides structured writing tools to help users organize and streamline their workflow.

1. Chapter Organization and Outline Management
2. Collaborative Writing
3. Content Customization and Personalization
4. Notes Creation
5. Version Control
6. Language Options (English/Urdu)
7. Character Development Window
8. Plot Development Window
9. Publishing Options

1.3.2 Module 2: AI-Analysis Tools

This module provides AI-assisted features to improve the writing process.

1. Fine-tuned AI-Assistant for Writing Suggestions
2. English Grammar Checker

1.4 User Classes and Characteristics

The following user classes are expected to interact with the VerseCraft application:

User Class	Description
Writers	Writers using the platform to create and manage their stories and novels. They will use the workspace dashboard, AI tools, and collaboration features to streamline their writing process.
Administrators	Personnel responsible for managing the platform's operation, ensuring system security, managing user accounts, and offering support when needed.
Developers	The technical team responsible for developing and maintaining the platform, fixing bugs, integrating new features, and ensuring the platform's functionality.
Testers	Individuals responsible for testing the application for bugs, usability issues, and ensuring the platform works as intended across different devices and environments.
Documentation Writers	Professionals tasked with creating user guides, tutorials, and other instructional content to help users and developers navigate the platform effectively.

Table 1.1: User Classes and Characteristics

Chapter 2

Project Requirements

This chapter describes the functional and non-functional requirements of the project.

2.1 Use-case/Event Response Table/Storyboarding

The selection of the requirement gathering technique(s) will depend on the type of project. For instance, the following use cases outline the key interactions:

1. **User Registration and Login**

Allows users to create an account, log in with their credentials, or reset their password.

2. **Create and View Writing Projects**

Writers can create new writing projects and view existing projects.

3. **Manage Chapters**

Users can create, edit, delete, and rearrange chapters within a project.

4. **Manage Notes**

Writers can add, edit, and delete notes for specific chapters or the entire project.

5. **Collaborative Writing**

Multiple users can collaborate on the same project with role-based access and real-time editing.

6. **View and Comment on Published Projects**

Readers can view published projects and leave comments or feedback.

7. **Manage Account Settings**

Users can update their profile, change email or password, and customize their preferences.

8. **AI-Assisted Grammar Check**

Writers can run grammar and spell checks on their projects using an AI-powered assistant.

9. **Manage Characters**

Writers can create, edit, and track character profiles within a project.

10. **Switch Language (English/Urdu)**

Users can toggle between English and Urdu for both the interface and content creation.

11. **Customize Workspace**

Writers can change the appearance of their workspace and adjust settings to optimize their writing environment.

12. **Export Project (PDF, DOCX, etc.)**

Allows users to export their entire project in various formats such as PDF or DOCX.

13. **Version Control**

Users can track changes, view the history of edits, and restore previous versions of their project.

14. **AI-Assisted Writing Assistant (ChatBot)**

Users can interact with an AI chatbot to get suggestions, guidance, or answers to questions during the writing process.

15. **Manage Project (Edit, Delete)**

Writers can edit the details of their project or delete it if necessary.

16. **View Analytics and Writing Metrics**

Provides writers with analytics on word count, session duration, chapter progression, and more.

17. **Publish Completed Project**

Allow writers to share their completed work with readers, publishers, or other . so others can view it, like, or comment on it.

18. **Manage Users and Projects (Admin)**

Allow Admin to manage users and projects to maintain system order and address user issues.

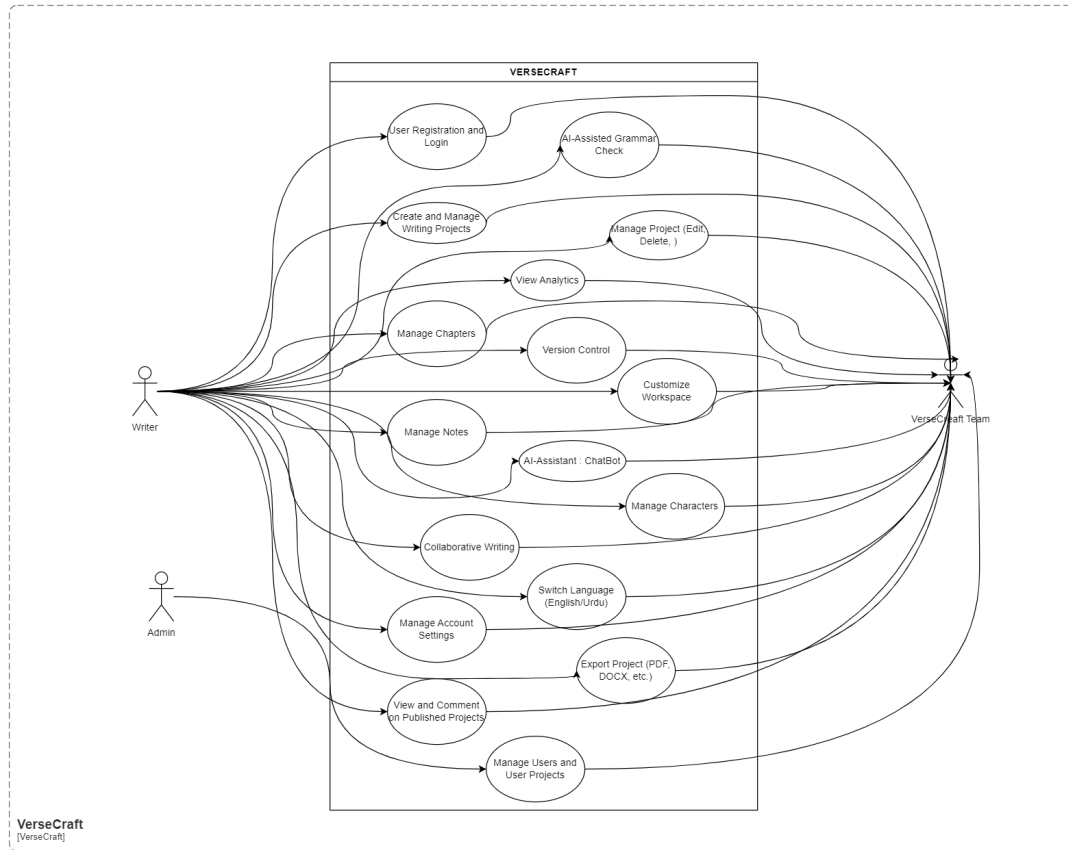


Figure 2.1: Use Case Diagram for VerseCraft

2.2 Detailed Use Cases

2.2.1 Use Case 1: User Registration and Login

Scope: VerseCraft

Level: User-Goal

Primary Actor: User

Stakeholders and Interests

- **Users:** Need a secure and straightforward way to create and access their accounts.
- **System Administrators:** Ensure the security and integrity of user data.
- **Developers and Designers:** Focus on providing a seamless and user-friendly authentication experience.

Preconditions

- User must have access to the internet.
- User must provide a valid email address.

Success Guarantee (Post Conditions)

- Users are successfully registered and can access their accounts.
- Users can securely reset their passwords if forgotten.

Main Success Scenario

1. User navigates to the registration page.
2. User enters required information (e.g., name, email, password).
3. System validates the input and creates a new user account.
4. System sends a confirmation email to the user.
5. User verifies their email and logs into their account.

Extensions

- **3a.** If the email is already in use:
 1. System notifies the user and prompts for a different email.
- **4a.** If the user forgets their password:
 1. User selects the 'Forgot Password' option.
 2. System sends a password reset link to the registered email.
 3. User resets the password using the link.

Special Requirements

- Ensure compliance with data protection regulations.
- Implement CAPTCHA to prevent automated registrations.

Technology and Data Variations List

- Support for social media logins (e.g., Google, Facebook).
- Multi-factor authentication for enhanced security.

Frequency of Occurrence

- High, as all new users must register and existing users must log in regularly.

2.2.2 Use Case 2: Create and Manage Writing Projects

Scope: VerseCraft

Level: User-Goal

Primary Actor: Writer

Stakeholders and Interests

- **Writers:** Interested in an efficient way to create, manage, and track their writing projects.
- **System Administrators:** Need to ensure that the system supports efficient data handling and user management.
- **Developers and Designers:** Focus on providing an intuitive and feature-rich environment for managing writing projects.

Preconditions

- User must be registered and logged in.

Success Guarantee (Post Conditions)

- New writing projects are created and saved correctly.

Main Success Scenario

1. Writer logs into their account.
2. Writer navigates to the project creation interface.

3. Writer selects the option to create a new project.
4. Writer inputs details for the new project, such as title, description, genre, and initial content.
5. System saves the new project and confirms its creation to the writer.

Extensions

- **3a.** If the writer selects an existing project:
 1. System displays project details.
- **4a.** If input data is invalid:
 1. System prompts for correct data.

Special Requirements

- None

Technology and Data Variations List

- Project data can be entered via web forms or direct text entry.
- Projects can be tagged with metadata such as genre, which may affect how they are handled or displayed by the system.

Frequency of Occurrence

- Frequent, as writers will regularly manage their writing projects.

2.2.3 Use Case 3: Manage Chapters

Scope: VerseCraft

Level: User-Goal

Primary Actor: Writer

Stakeholders and Interests

- **Writers:** Interested in flexible and intuitive tools to structure their stories via chapter management.
- **Editors:** Seek streamlined processes for revising and organizing content.
- **System Administrators:** Ensure robust functionality and user-friendly interfaces for managing content.

Preconditions

- User must be registered, logged in, and must have an existing project.

Success Guarantee (Post Conditions)

- Chapters within a project are correctly created, modified, deleted, or rearranged.
- Changes are saved and accurately reflected in the project.

Main Success Scenario

1. Writer logs into their account and opens an existing project.
2. Writer accesses the chapter management interface within the project.
3. Writer selects to add a new chapter, providing a title and initial content.
4. System adds the chapter to the project at the specified location.
5. Writer edits the content of any chapter or rearranges chapters as needed.
6. System saves changes and updates the project layout.
7. Writer can delete any chapter, with the system confirming deletion and updating the project accordingly.

Extensions

- **3a.** If the writer edits an existing chapter:
 1. System loads the chapter's content for editing.
 2. Writer modifies content and saves changes.

- **5a.** If input data during editing is invalid:
 1. System prompts to correct the data.
- **6a.** If the writer wants to rearrange chapters:
 1. System allows drag-and-drop functionality to reorder chapters.
 2. System confirms the new order.
- **7a.** If the writer deletes a chapter:
 1. System asks for confirmation before deletion.
 2. Upon confirmation, the chapter is deleted and remaining chapters are adjusted.

Special Requirements

- None

Technology and Data Variations List

- Chapters can be edited through a text editor interface that supports rich text formatting.
- Rearranging chapters utilizes a drag-and-drop interface.

Frequency of Occurrence

- Regular, as managing chapters is a fundamental part of structuring a writing project.

2.2.4 Use Case 4: Manage Notes (Inline Notes)

Scope: VerseCraft

Level: User-Goal

Primary Actor: Writer

Stakeholders and Interests

- **Writers:** Need efficient ways to annotate their manuscripts for better organization and referencing.
- **Editors:** Benefit from clear, accessible notes that enhance the editing process.

- **System Administrators:** Ensure the notes feature is seamless and integrated effectively with the rest of the writing environment.

Preconditions

- User must be logged in and have an existing project with at least one chapter.

Success Guarantee (Post Conditions)

- Notes are accurately created, modified, or deleted within the project.
- Changes to notes are saved and can be referenced or removed as needed.

Main Success Scenario

1. Writer accesses an existing project and selects a chapter.
2. Writer chooses to add a new note linked to a specific part of the chapter text.
3. Writer enters the note content and saves it, which is then displayed inline or as a margin note.
4. Writer edits existing notes by selecting them and modifying the content.
5. System saves the updated notes.
6. Writer can delete any note, with the system confirming the deletion and removing the note.

Extensions

- **3a.** If the writer inputs an excessively long note:
 1. System prompts to shorten the note to meet character limits.
- **4a.** If the writer attempts to save an empty note:
 1. System prompts to add content before saving.
- **6a.** If the writer deletes a note:
 1. System asks for confirmation to prevent accidental deletions.

Special Requirements

- None

Technology and Data Variations List

- Notes are text-based but may support basic formatting to distinguish them from the main text.
- The interface for notes might offer options for color coding or tagging for better organization.

Frequency of Occurrence

- Frequent, as notes are a critical part of the writing and editing process.

2.2.5 Use Case 5: Collaborative Writing

Scope: VerseCraft

Level: User-Goal

Primary Actor: Collaborative Writers

Stakeholders and Interests

- **Collaborative Writers:** Interested in a seamless and synchronized writing experience with other authors.
- **Project Manager:** Oversees the collaborative process, ensuring all participants can contribute effectively.
- **System Administrators:** Ensure the system supports multiple users simultaneously without performance degradation.

Preconditions

- All users must be registered and have permissions set for the project.

Success Guarantee (Post Conditions)

- Multiple users can access the project simultaneously.

- Changes by one user are visible in real-time to all others.
- Role-based permissions are respected and enforced.

Main Success Scenario

1. Project creator invites collaborators to the project and assigns roles.
2. Collaborators log into the system and access the shared project.
3. Users make real-time edits to the project, with changes instantly visible to all participants.
4. System continuously saves the project state, ensuring no data loss.
5. Collaborators use communication tools integrated within the platform to discuss and coordinate efforts.
6. Project manager reviews roles and access permissions periodically to ensure proper collaboration flow.

Extensions

- **1a.** If an invitee does not have an account:
 1. Invitee is prompted to register before accessing the project.
- **3a.** If conflicts arise from simultaneous edits:
 1. System flags the conflict and prompts users to resolve it.
- **5a.** If a user encounters system performance issues:
 1. System admin is notified to troubleshoot and optimize performance.

Special Requirements

- None

Technology and Data Variations List

- Edits are made through a web-based text editor capable of handling simultaneous inputs.
- Role-based access control system to manage permissions for viewing, editing, and administrative actions.

Frequency of Occurrence

- Regular, especially in projects involving multiple authors or large-scale collaborative efforts.

2.2.6 Use Case 6: View and Comment on Published Projects

Scope: VerseCraft

Level: User-Goal

Primary Actor: Reader

Stakeholders and Interests

- **Readers:** Interested in accessing published content and engaging with it through comments or feedback.
- **Authors:** Seek feedback to improve future writings or engage with their audience.
- **System Administrators:** Ensure the system facilitates smooth interaction between readers and content.

Preconditions

- The project must be published and set to allow public or restricted access.

Success Guarantee (Post Conditions)

- Readers can view published projects as intended by the author.
- Comments are properly submitted and displayed under the projects.

Main Success Scenario

1. Reader navigates to the public or shared link of a published project.
2. System displays the project content along with existing comments.
3. Reader reads the content and decides to leave feedback.
4. Reader enters a comment in the designated comment section.
5. System validates and posts the comment under the project.

6. Other readers and the author can view and respond to the comment.

Extensions

- **4a.** If the reader attempts to post an inappropriate comment:
 1. System filters the comment and warns the reader about content guidelines.
- **4b.** If the reader is not logged in but attempts to comment:
 1. System prompts the reader to log in or register.
- **5a.** If there is an error while posting the comment:
 1. System displays an error message and suggests the reader to try again.

Special Requirements

- None

Technology and Data Variations List

- Comments may be moderated automatically or manually to maintain quality and appropriateness.
- Support for notifications to authors when new comments are posted on their projects.

Frequency of Occurrence

- Frequent, as viewing and commenting are common interactions for published projects.

2.2.7 Use Case 7: Manage Account Settings

Scope: VerseCraft

Level: User-Goal

Primary Actor: User

Stakeholders and Interests

- **Users:** Interested in maintaining control over their personal information and preferences for a tailored experience.

- **System Administrators:** Ensure that account settings are manageable and secure, adhering to privacy policies.
- **Security Team:** Focus on maintaining the integrity and security of user data during updates.

Preconditions

- User must be logged in.

Success Guarantee (Post Conditions)

- User's profile, email, password, and preferences are updated accurately.
- System confirms all changes to the user.

Main Success Scenario

1. User logs into their account and navigates to the account settings page.
2. User selects the option to update their profile information and enters new details.
3. System validates and saves the new profile information.
4. User chooses to change their email or password and provides the new data.
5. System verifies the changes with the user, possibly through confirmation emails or security questions.
6. User adjusts their preferences for notifications, language, or interface settings.
7. System saves these preferences and applies them immediately to the user's experience.

Extensions

- **4a.** If the user enters an invalid email or password:
 1. System displays an error and requests correct format or security criteria.
- **5a.** If the user must verify changes (e.g., email change):
 1. System sends a verification link to the new email.
 2. User must click the verification link to complete the update.

- **7a.** If the user chooses conflicting preferences:
 1. System prompts the user to resolve conflicts before saving.

Special Requirements

- None

Technology and Data Variations List

- Account settings changes might include dynamic forms that adapt based on user data and previous settings.
- Multi-factor authentication could be required for sensitive changes like email or password updates.

Frequency of Occurrence

- Occasional, as users typically set preferences infrequently but may update profiles or security settings more regularly.

2.2.8 Use Case 8: AI-Assisted Grammar Check

Scope: VerseCraft

Level: User-Goal

Primary Actor: Writer

Stakeholders and Interests

- **Writers:** Seek efficient tools for improving grammatical accuracy and readability of their texts.
- **Editors:** Benefit from reduced preliminary editing workload due to pre-checked texts.
- **Developers and Designers:** Focus on integrating and maintaining AI technologies that support real-time text analysis.

Preconditions

- User must be logged into their account and must have a text drafted in a project.

Success Guarantee (Post Conditions)

- The text is checked for grammatical and spelling errors.
- Suggestions for corrections are provided to the user.

Main Success Scenario

1. Writer accesses a draft within a project.
2. Writer selects the option to run a grammar and spell check on the text.
3. AI-powered assistant analyzes the text for any grammatical or spelling mistakes.
4. System displays a report of issues found, with suggestions for corrections.
5. Writer reviews the suggestions and chooses to accept or ignore them.
6. System updates the text based on the writer's selections.

Extensions

- **4a.** If no errors are found:
 1. System informs the writer that the text has no detectable errors.
- **5a.** If the writer disagrees with a suggestion:
 1. Writer can choose to ignore the suggestion and the text remains unchanged.
- **6a.** If the writer requests further explanation on a correction:
 1. System provides additional context or rules explaining the correction.

Special Requirements

- None

Technology and Data Variations List

- The grammar checking might be supported by cloud-based AI services for continuous learning and updates.
- The interface allows writers to interact with corrections through tooltips or an interactive sidebar.

Frequency of Occurrence

- Frequent, as writers regularly need to ensure the quality of their texts.

2.2.9 Use Case 9: Manage Characters

Scope: VerseCraft

Level: User-Goal

Primary Actor: Writer

Stakeholders and Interests

- **Writers:** Interested in detailed and easily accessible character management tools to enhance story development.
- **Editors:** Benefit from understanding characters deeply to provide more targeted feedback on manuscripts.
- **System Administrators:** Ensure that the character management feature is robust and integrates seamlessly with other project features.

Preconditions

- User must be logged in and must have an existing project.

Success Guarantee (Post Conditions)

- Characters within a project are correctly created, modified, or archived.
- Character profiles are accessible and up-to-date as per the writer's inputs.

Main Success Scenario

1. Writer accesses a project and navigates to the character management section.
2. Writer selects the option to create a new character profile.
3. Writer inputs details such as name, age, description, background, and motivations.
4. System saves the new character profile within the project.
5. Writer edits existing character profiles, updating information as the story develops.

6. System saves the changes to the profiles.
7. Writer can archive characters that are no longer active in the story but retain their profiles for reference.

Extensions

- **3a.** If the writer inputs incomplete or incorrect character details:
 1. System prompts the writer to complete all required fields or correct information.
- **5a.** If changes conflict with story developments:
 1. System alerts the writer to inconsistencies requiring resolution.
- **7a.** If the writer wants to reactivate an archived character:
 1. System provides an option to restore the character to active status.

Special Requirements

- None

Technology and Data Variations List

- Character profiles may support multimedia elements like images or voice notes for deeper character development.
- Integration with plot and chapter tools to reflect character developments directly in the manuscript.

Frequency of Occurrence

- Regular, as character development is a continuous process throughout the lifecycle of a writing project.

2.2.10 Use Case 10: Switch Language (English/Urdu)

Scope: VerseCraft

Level: User-Goal

Primary Actor: User

Stakeholders and Interests

- **Users:** Interested in utilizing the platform in their preferred language to enhance understanding and usability.
- **Content Creators:** Need to produce content in multiple languages to reach a broader audience.
- **System Administrators:** Ensure seamless language switching functionality and support for multilingual content management.

Preconditions

- User must be logged in.

Success Guarantee (Post Conditions)

- The user interface language changes according to user selection.
- Content creation tools adapt to the selected language, affecting spelling and grammar checks.

Main Success Scenario

1. User accesses the language settings from their account or dashboard.
2. User selects either English or Urdu from the language options.
3. System applies the language change to the user interface immediately.
4. If the user is working on a project, the content creation tools switch to the selected language, including relevant dictionaries and grammar rules.
5. System confirms the switch and continues to operate in the selected language for all functionalities.

Extensions

- **2a.** If the system encounters an error while switching languages:
 1. System displays an error message and asks the user to try again.
- **4a.** If the user switches language while editing content:
 1. System prompts the user to save changes before the switch takes effect.

Special Requirements

- None

Technology and Data Variations List

- Implementation may include dynamic loading of language-specific resources like UI text, help documentation, and tooltips.
- Language settings should be user-specific and persist across sessions.

Frequency of Occurrence

- Occasional, as users may not frequently switch languages but need the functionality accessible when required.

2.2.11 Use Case 11: Customize Workspace

Scope: VerseCraft

Level: User-Goal

Primary Actor: Writer

Stakeholders and Interests

- **Writers:** Interested in a personalized and comfortable writing environment that enhances productivity and creativity.
- **System Designers:** Ensure the workspace customization features are user-friendly and meet various user needs.
- **System Administrators:** Maintain system performance and compatibility with various customization options.

Preconditions

- User must be logged in and have an active project.

Success Guarantee (Post Conditions)

- The writer's workspace settings are customized according to their preferences.

- Changes are applied immediately and persist during future sessions.

Main Success Scenario

1. Writer accesses the customization settings from their project dashboard.
2. Writer selects from various themes and layouts to change the visual appearance of the workspace.
3. Writer adjusts text size, font style, and color scheme to suit their visual preferences.
4. Writer configures toolbars and panels for easier access to frequently used tools.
5. System applies and saves the customization settings.
6. Writer resumes work with the new settings enhancing their writing experience.

Extensions

- **2a.** If the writer selects a theme or layout that is not compatible with certain features:
 1. System notifies the writer about the incompatibility and suggests alternatives.
- **3a.** If there is a request to save a custom theme:
 1. System allows the writer to name and save the custom configuration for future use.
- **5a.** If there is an error applying the settings:
 1. System displays an error message and asks the writer to retry.

Special Requirements

- None

Technology and Data Variations List

- Customization options might include predefined themes or allow for more granular adjustments like manual color selection.
- Settings interface could include a preview feature, allowing writers to see changes in real-time before applying them.

Frequency of Occurrence

- Occasionally, as writers set up their environment according to personal preference and might not change settings frequently after initial setup.

2.2.12 Use Case 12: Export Project (PDF, DOCX, etc.)

Scope: VerseCraft

Level: User-Goal

Primary Actor: Writer

Stakeholders and Interests

- **Writers:** Need to share or archive projects in universally accepted formats.
- **Editors:** Require access to project files in editable or reviewable formats to provide feedback.
- **System Administrators:** Ensure the export functionality is reliable and supports multiple file formats.

Preconditions

- User must be logged in and have a project ready for export.

Success Guarantee (Post Conditions)

- The project is exported in the selected format.
- The exported file maintains the integrity of content and formatting.

Main Success Scenario

1. Writer accesses the project they wish to export.
2. Writer selects the 'Export' option from the project menu.
3. Writer chooses the format for export, such as PDF or DOCX.
4. System processes the project into the selected format, preserving all textual and formatting details.

5. System provides the writer with a download link or automatically starts the download.
6. Writer saves the file to their local machine or cloud storage.

Extensions

- **3a.** If the writer selects a format not supported by the current content:
 1. System notifies the writer and suggests compatible formats.
- **5a.** If the export process fails:
 1. System alerts the writer and offers troubleshooting steps or the option to retry.

Special Requirements

- None

Technology and Data Variations List

- Support for exporting to additional formats such as ePub or HTML may be included.
- Real-time preview of the document in the chosen format before finalizing the export.

Frequency of Occurrence

- Occasional, as exporting is typically done towards the completion of writing projects or for specific distribution purposes.

2.2.13 Use Case 13: Version Control

Scope: VerseCraft

Level: User-Goal

Primary Actor: Writer

Stakeholders and Interests

- **Writers:** Interested in maintaining and tracking revisions to ensure they can revert or reference past versions.
- **Editors:** Benefit from accessing different versions to understand the evolution of the text and provide contextual feedback.
- **System Administrators:** Ensure that version control mechanisms are efficient and data integrity is maintained throughout revisions.

Preconditions

- User must be logged in and have an active project with version tracking enabled.

Success Guarantee (Post Conditions)

- Changes to the project are logged and can be reviewed by the user.
- Users can revert to previous versions of the project as needed.

Main Success Scenario

1. Writer makes changes to a project, which are automatically saved as a new version.
2. Writer wants to view the history of edits and accesses the version history feature.
3. System displays a list of all versions with timestamps and summaries of changes.
4. Writer selects a specific version to review the detailed content of that iteration.
5. If needed, writer can choose to revert the project to a selected past version.
6. System restores the project to the chosen version, while still preserving newer versions for potential future use.

Extensions

- **2a.** If the version history is not enabled:
 1. System prompts the writer to enable version tracking for future changes.
- **4a.** If the writer selects a version with substantial differences:

1. System may provide a side-by-side comparison to highlight changes.
- **5a.** If the writer reverts to a previous version:
 1. System asks for confirmation before making the change to avoid accidental rollbacks.

Special Requirements

- None

Technology and Data Variations List

- Version control might integrate with cloud storage services to enhance data backup and recovery.
- Implementations could include graphical interfaces to visually compare differences between versions.

Frequency of Occurrence

- Regular, as tracking changes is a crucial part of managing a dynamic and evolving writing project.

2.2.14 Use Case 14: AI-Assisted Writing Assistant (ChatBot)

Scope: VerseCraft

Level: User-Goal

Primary Actor: Writer

Stakeholders and Interests

- **Writers:** Seek real-time assistance and suggestions to enhance their writing quality and efficiency.
- **Editors:** Benefit from writers using the assistant to improve initial drafts before formal review.
- **AI Developers:** Focus on refining the chatbot's capabilities to better serve user needs and provide accurate, contextually relevant advice.

Preconditions

- User must be logged in and actively working within a project.

Success Guarantee (Post Conditions)

- Users receive relevant suggestions, guidance, or answers from the AI chatbot.
- Interactions with the chatbot enhance the user's writing process and project quality.

Main Success Scenario

1. Writer initiates a session with the AI chatbot through their project interface.
2. Writer asks specific questions related to their writing, such as plot development, character traits, or grammar.
3. AI chatbot analyzes the query and the context provided by the current content of the project.
4. AI chatbot provides suggestions, tips, or corrections relevant to the writer's query.
5. Writer evaluates the chatbot's feedback and decides to implement the advice or explore further interactions.
6. System logs the interaction to refine future responses and improve the AI model.

Extensions

- **2a.** If the query is beyond the AI's current capabilities:
 1. Chatbot admits its limitations and may suggest external resources or the option to seek human assistance.
- **4a.** If the writer disagrees with the suggestions:
 1. Writer can request alternative advice or clarify their question for more accurate support.
- **5a.** If the writer frequently uses the chatbot effectively:
 1. System adjusts to better align with the writer's style and preferences over time.

Special Requirements

- None

Technology and Data Variations List

- The chatbot could utilize advanced natural language processing (NLP) technologies to understand and generate human-like responses.
- Integration with the project's content management system to allow the chatbot to access relevant project details and provide contextual advice.

Frequency of Occurrence

- Frequent, as writers often need assistance throughout various stages of the writing process.

2.2.15 Use Case 15: Manage Project (Edit, Delete)

Scope: VerseCraft

Level: User-Goal

Primary Actor: Writer

Stakeholders and Interests

- **Writers:** Need flexibility in managing the details and existence of their writing projects.
- **System Administrators:** Ensure the integrity and efficient management of project data.
- **Data Backup and Recovery Teams:** Concerned with the safe storage and potential recovery of deleted projects.

Preconditions

- User must be logged in and must have at least one existing project.

Success Guarantee (Post Conditions)

- Project details are updated according to the user's modifications.
- Projects are deleted as requested, with appropriate confirmations and the possibility of recovery if needed.

Main Success Scenario

1. Writer accesses the project management section within their account.
2. Writer selects a project to edit or delete.
3. For editing, writer modifies details such as project title, description, settings, or access permissions.
4. System confirms the changes and updates the project accordingly.
5. For deletion, writer confirms their intention to delete the project.
6. System provides a secondary confirmation prompt to prevent accidental deletions.
7. Upon confirmation, system deletes the project and notifies the writer of the successful deletion.

Extensions

- **3a.** If the writer attempts to edit a project while another user is also making changes (in collaborative projects):
 1. System manages the concurrent edits or locks the project temporarily to prevent overwrite conflicts.
- **6a.** If the writer decides against deletion after reaching the confirmation prompt:
 1. System cancels the deletion process and retains the project.
- **7a.** If the system offers a recovery option for deleted projects:
 1. Writer is informed about the recovery process and duration during which the project can be restored.

Special Requirements

- None

Technology and Data Variations List

- Project editing could include more detailed adjustments like role assignments for collaborative projects or privacy settings.
- Deletion processes might include automatic archival for a set period before permanent deletion to allow for recovery.

Frequency of Occurrence

- Occasional, as project editing is common but deletions are less frequent, typically occurring when a project is no longer needed or requires reevaluation.

2.2.16 Use Case 16: View Analytics and Writing Metrics

Scope: VerseCraft

Level: User-Goal

Primary Actor: Writer

Stakeholders and Interests

- **Writers:** Interested in tracking their productivity and progress to enhance their writing habits and output.
- **Project Managers:** Utilize analytics to oversee project timelines and writer efficiency.
- **System Administrators:** Ensure the analytics features are accurate, timely, and supportive of user goals.

Preconditions

- User must be logged in and have active writing projects.

Success Guarantee (Post Conditions)

- Analytics and metrics are accurately displayed, reflecting the latest writing activities.
- Writers gain insights that could lead to improved writing strategies and project management.

Main Success Scenario

1. Writer accesses the analytics feature from their project dashboard.
2. System displays various metrics such as total word count, session duration, daily goals, chapter completion status, and recent activity.
3. Writer reviews the metrics to assess their progress and identify areas needing improvement.
4. Writer uses this data to plan future writing sessions or adjust project timelines.
5. System updates metrics in real-time as the writer continues to work on the project.

Extensions

- **2a.** If the analytics feature encounters an error loading data:
 1. System displays an error message and offers to retry fetching the data.
- **3a.** If the writer wants more detailed analytics:
 1. Writer can customize the dashboard to include additional metrics such as reading ease, passive voice usage, or frequent word use.
- **5a.** If the writer adjusts their project based on analytics:
 1. System recalculates predictions for project completion and updates the timeline accordingly.

Special Requirements

- None

Technology and Data Variations List

- Analytics might be customizable, allowing writers to select which metrics are displayed on their dashboard.
- Integration with machine learning models to provide predictive insights based on past performance and current trends.

Frequency of Occurrence

- Regular, as ongoing project work and daily writing sessions generate continuous data for analysis.

2.2.17 Use Case 17: Publish Completed Project

Scope: VerseCraft

Level: User-Goal

Primary Actor: Writer

Stakeholders and Interests

- **Writers:** Want to share their completed work with readers, publishers, or other platforms.
- **Readers:** Interested in accessing and reading published content.
- **Community Managers:** Need to ensure published content adheres to community guidelines.

Preconditions

- User must be logged in and have a completed project ready for publication.

Success Guarantee (Post Conditions)

- The project is successfully published to the chosen platform.
- Readers can access and engage with the published content.

Main Success Scenario

1. Writer navigates to the completed project and selects the option to publish.
2. Writer chooses the publishing options, such as public, private, or community-specific visibility.
3. Writer reviews and confirms the content to be published.
4. System verifies compliance with guidelines and publishes the project.

5. Published content becomes available on the designated platform for readers to access.
6. System notifies the writer of the successful publication and provides a shareable link.

Extensions

- **2a.** If the writer wants to publish to multiple platforms:
 1. Writer selects additional platforms, and system confirms compatibility.
- **4a.** If the content does not comply with guidelines:
 1. System highlights the sections requiring modification before publication can proceed.
- **5a.** If there are connectivity issues during publication:
 1. System saves the publication attempt and retries when connectivity is restored.

Special Requirements

- Ensure compliance with publishing guidelines and data privacy requirements.
- Provide a preview option for writers before finalizing publication.

Technology and Data Variations List

- Publishing could include options for different file formats, such as PDF, EPUB, or HTML.
- Integration with external publishing platforms like Amazon Kindle or IngramSpark may be available.

Frequency of Occurrence

- Occasional, as publishing occurs only when a project reaches its final state.

2.2.18 Use Case 18: Manage Users and Projects (Admin)

Scope: VerseCraft

Level: User-Goal

Primary Actor: Administrator

Stakeholders and Interests

- **Administrators:** Need to manage users and projects to maintain system order and address user issues.
- **Users:** Expect their accounts and projects to be managed securely and efficiently.

Preconditions

Administrator is logged into the admin panel.

Success Guarantee (Post Conditions)

User accounts and projects are managed effectively and securely.

Main Success Scenario

1. Administrator navigates to the relevant management section (Users or Projects).
2. Administrator performs actions like creating, modifying, deleting, or viewing details.
3. Changes are saved and logged.

Extensions

- **2a.** Deleting a user: System prompts for confirmation.
- **2b.** Resolving project conflicts: System provides tools for comparison and communication.

Special Requirements

Secure access control (e.g., multi-factor authentication) to the admin panel.

Technology and Data Variations List

None specified.

Frequency of Occurrence

Regular, as needed for system maintenance and user support.

2.3 Functional Requirements

This section describes the functional requirements of the system in a natural language style. The functional requirements are organized by modules, reflecting the specific features of each system component.

2.3.1 Module 1: Workspace Dashboard

Following are the requirements for the Workspace Dashboard module:

1. Chapter Organization and Outline Management:

- Users can manage their chapters separately, focusing on their content.
- Chapters can be reordered using a drag-and-drop feature, allowing users to restructure their documents.

2. Collaborative Writing:

- Multiple users can collaborate on the same project, with different project roles and responsibilities.
- Changes will be fully synced in real-time for all collaborators.

3. Content Customization and Personalization:

- Users can customize their project settings, such as defining themes and content styles, for a more personalized experience.

4. Notes Creation:

- Users can create and organize notes alongside their written content, making the revision and editing process smoother.

5. Version Control:

- Users can manage and track different project versions, allowing them to return to previous versions or compare changes to maintain content history.

6. Language Options (English/Urdu):

- Users can create content in either English or Urdu, with dashboard elements changing according to the selected language.

7. Character Development Window:

- Users can create and manage characters, ensuring coherence with project content.
- Options for character design and integration into the story workflow will be available.

8. Plot Development Window:

- Users can create and manage main plots and subplots, customizing their design to fit the story easily.

9. Publishing Options:

- Users can publish their content with different privacy options (e.g., private or public).

2.3.2 Module 2: AI-Analysis Tools

Following are the requirements for the AI-Analysis Tools module:

1. Fine-tuned AI-Assistant:

- The system will provide an AI language model specifically designed to enhance written content by improving flow and coherence.

2. English Grammar Checker:

- A highly accurate grammar-checking tool will identify errors and improve the overall readability of the content.

2.4 Non-Functional Requirements

This section specifies non-functional requirements, focusing on qualities like reliability, usability, performance, and security.

2.4.1 Reliability

The system must minimize failures and maintain high reliability. Metrics such as Mean Time Between Failures (MTBF) will be used to measure reliability. Measures should be in place to detect and correct errors, specifying the consequences of failure and strategies to mitigate them.

2.4.2 Usability

Usability requirements include ease of learning, ease of use, error avoidance, and recovery. The requirements are aimed at optimizing the user experience and ensuring accessibility for all users. For example:

- **USE-1:** The system shall allow users to retrieve their previous work with a single interaction.

2.4.3 Performance

Specific performance requirements will be outlined for various system operations. For example:

- **PER-1:** 95% of webpages generated by the system shall load completely within 4 seconds for users with a 20 Mbps or faster Internet connection.

2.4.4 Security

The system must protect against unauthorized access, with security measures quantified by factors such as effort, skill level, and time required to breach security. The requirements should avoid specific implementation details like passwords.

2.5 Domain Model

A representation of the domain model will be created to illustrate the core components, entities, and their relationships within the system.

[2.2.](#)

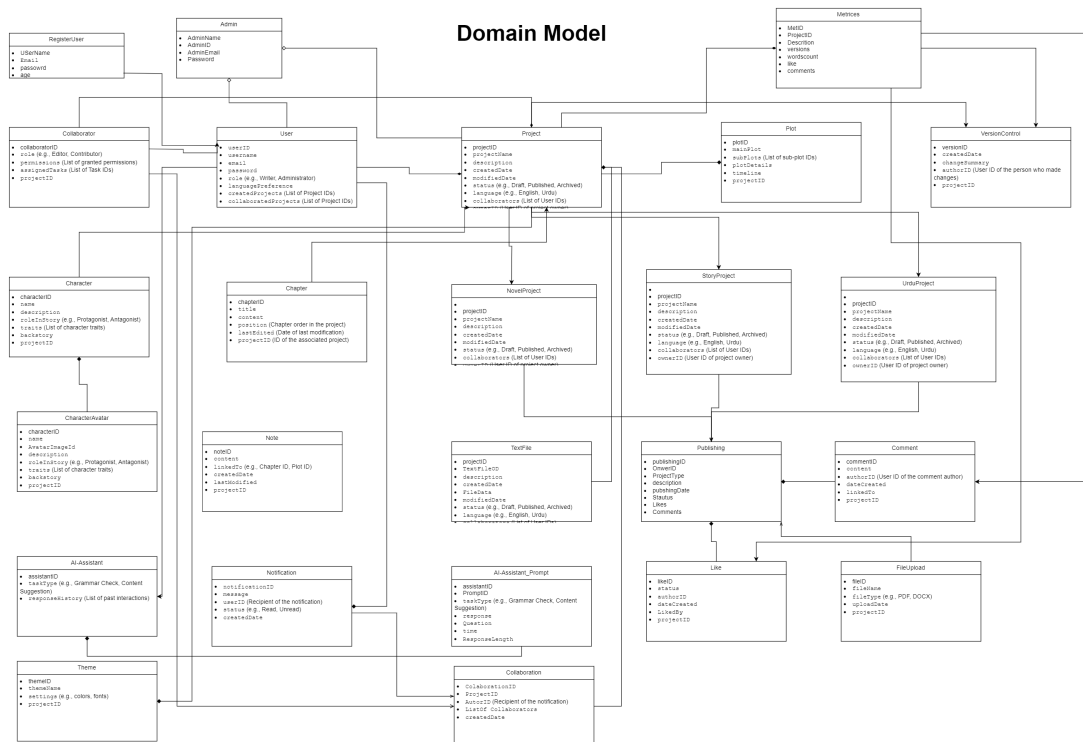


Figure 2.2: Domain Model of the System

Chapter 3

System Overview

This chapter provides a comprehensive overview of the VerseCraft system, detailing its functionality, context, and design. It outlines the architectural framework, design models, and data structures that underpin the application, ensuring a robust and scalable solution for managing the creative writing process.

3.1 Architectural Design

3.1.1 Modular Program Structure

VerseCraft is architected using a **Client-Server** model, leveraging a **Multi-tiered** architecture to ensure scalability, maintainability, and efficient performance. The system is decomposed into distinct modules, each responsible for specific functionalities, enabling seamless collaboration and integration.

3.1.1.1 Architecture Layers

1. Presentation Layer (Frontend):

- **Technology:** React.js
- **Responsibilities:**
 - Renders the user interface.
 - Handles user interactions and input.
 - Communicates with the backend through API calls.
- **Components:**
 - Workspace Dashboard

- AI-Analysis Tools Interface
- User Authentication Pages

2. Application Layer (Backend):

- **Technology:** Node.js with Express.js
- **Responsibilities:**
 - Processes business logic.
 - Manages API endpoints.
 - Handles data validation and processing.
- **Components:**
 - User Management Module
 - Project Management Module
 - AI Integration Module

3. Data Layer:

- **Technology:** MongoDB
- **Responsibilities:**
 - Stores and retrieves data.
 - Manages database schemas and relationships.
- **Components:**
 - User Data Storage
 - Project and Content Storage
 - AI Model Data Storage

4. AI Layer:

- **Technology:** Hugging Face, Flask, PyTorch/TensorFlow
- **Responsibilities:**
 - Provides AI-assisted functionalities.
 - Handles grammar checking and writing suggestions.
- **Components:**
 - AI-Assistant API
 - Grammar Checker Service

3.1.1.2 Module Interactions

- **Frontend ↔ Backend:** The frontend communicates with the backend via RESTful APIs, sending user inputs and receiving processed data or responses.
- **Backend ↔ AI Layer:** The backend interfaces with the AI layer to utilize AI-driven features, such as grammar checking and writing assistance.
- **Backend ↔ Data Layer:** The backend manages data transactions, ensuring data integrity and consistency within the MongoDB database.

3.1.1.3 High-Level System Diagram

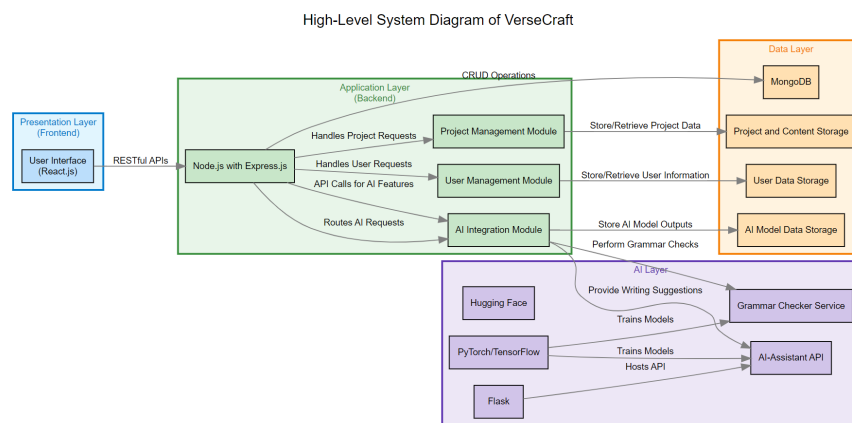


Figure 3.1: High-Level System Diagram of VerseCraft

3.1.1.4 Detailed Architecture Style: MVC (Model-View-Controller)

- **Model:** Represents the data structure (MongoDB schemas for Users, Projects, Chapters, etc.).
- **View:** The user interface built with React.js.
- **Controller:** Backend services in Node.js handling requests, processing data, and interfacing with the AI and Data layers.

This MVC pattern ensures a clear separation of concerns, facilitating easier maintenance and scalability.

3.2 Design Models

Design models are essential for visualizing and planning the system's structure and behavior. Below are the key design models applicable to VerseCraft, utilizing an Object-Oriented Development Approach.

3.2.1 Activity Diagram

Purpose: Illustrates the workflow of major functionalities within VerseCraft, such as creating a writing project, managing chapters, and utilizing AI-assisted tools.

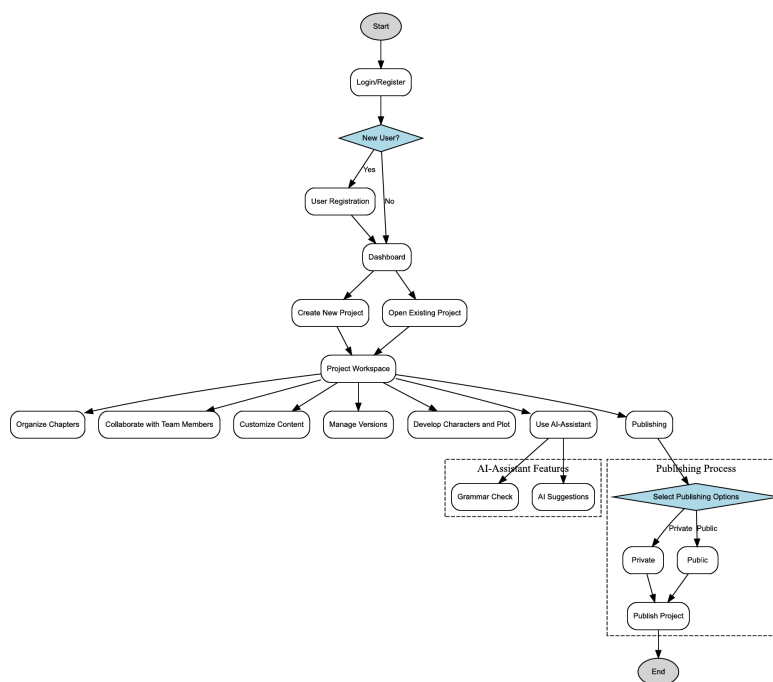


Figure 3.2: Activity Diagram for VerseCraft

Description:

- **Start:** User logs into the system.
- **Main Activities:**
 - Create/Manage Writing Projects
 - Organize Chapters
 - Collaborate with Other Writers
 - Utilize AI-Analysis Tools

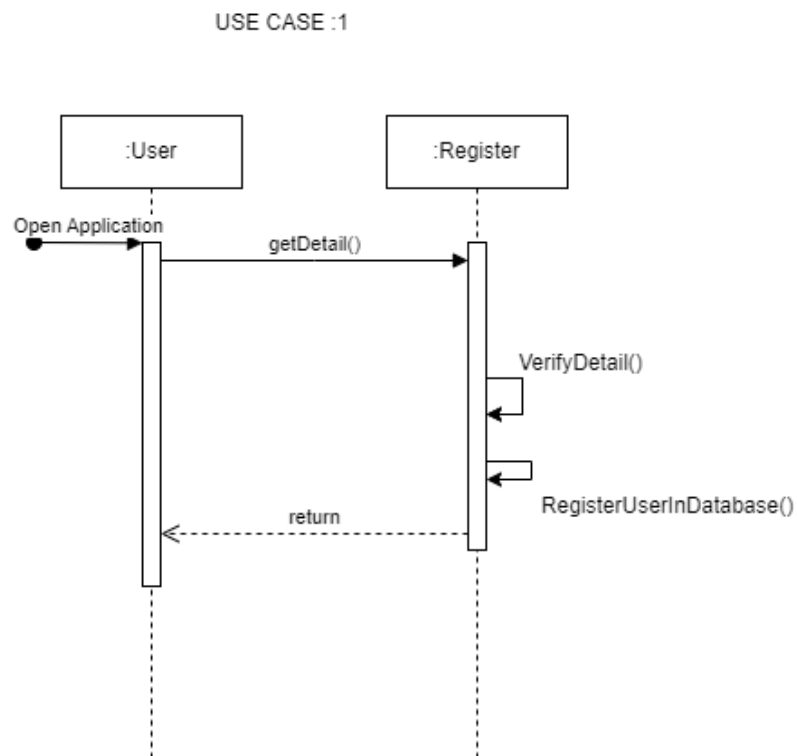


Figure 3.4: Sequence Diagram for User Registration and Login

3.2.3.2 Use Case 2: Create and View Writing Projects

Description: Writers can create new writing projects and view existing projects.

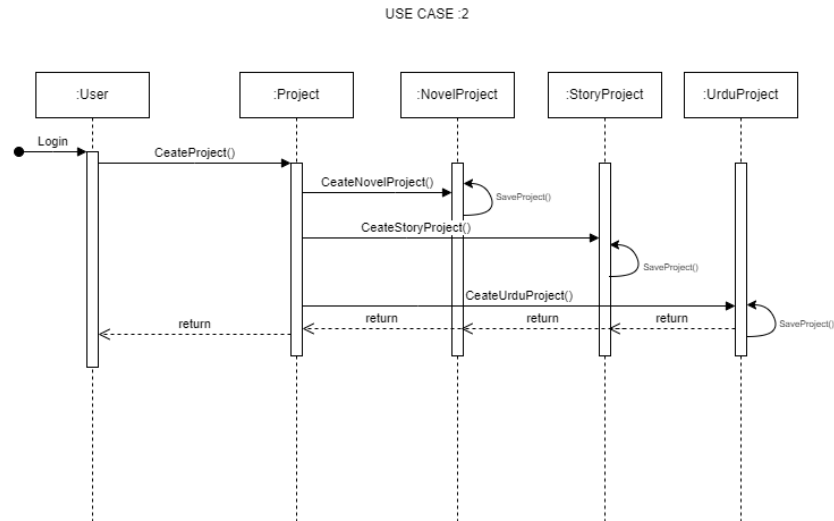


Figure 3.5: Sequence Diagram for Create and View Writing Projects

3.2.3.3 Use Case 3: Manage Chapters

Description: Users can create, edit, delete, and rearrange chapters within a project.

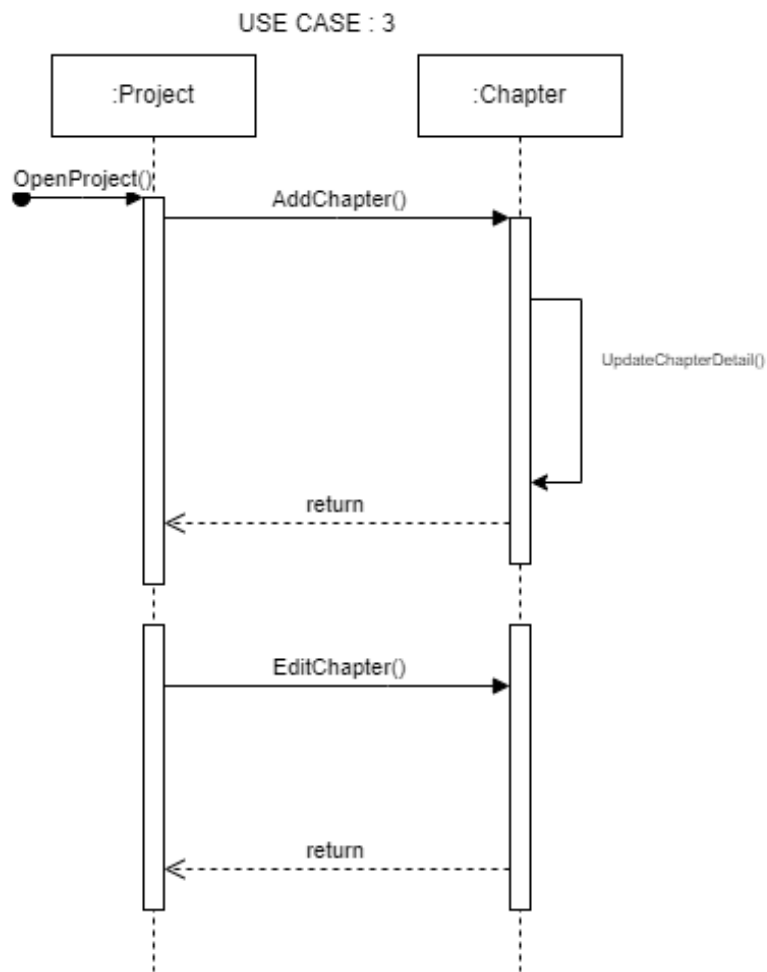


Figure 3.6: Sequence Diagram for Manage Chapters

3.2.3.4 Use Case 4: Manage Notes

Description: Writers can add, edit, and delete notes for specific chapters or the entire project.

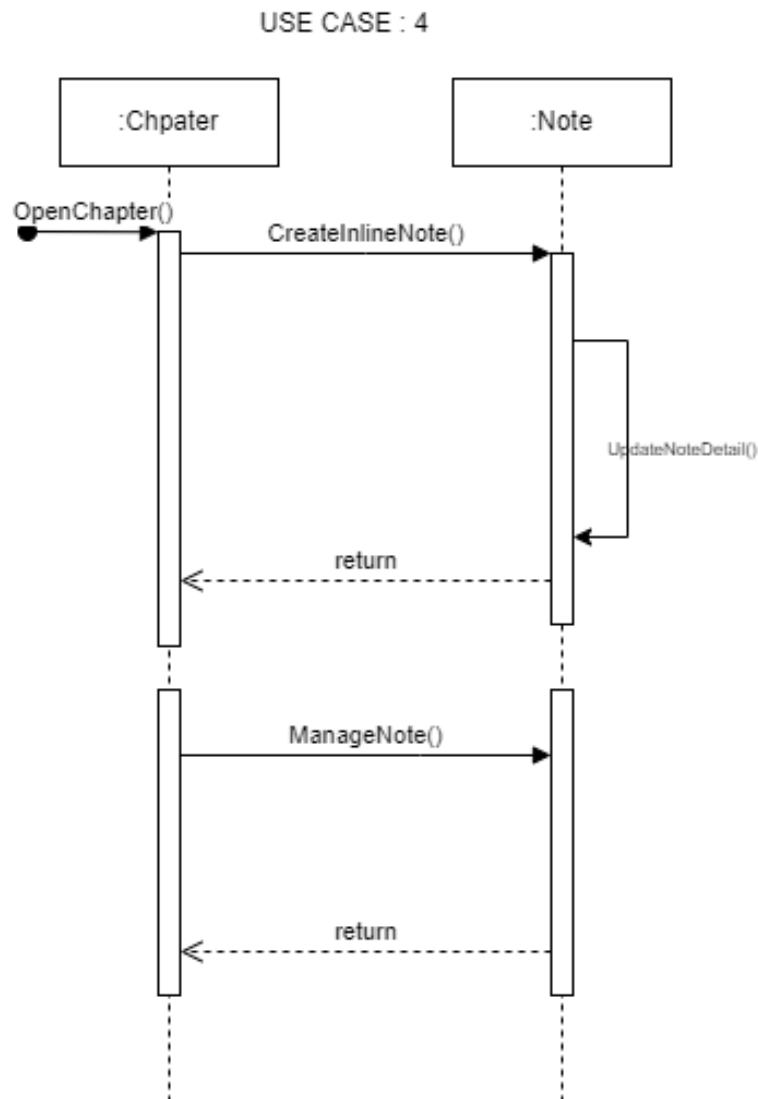


Figure 3.7: Sequence Diagram for Manage Notes

3.2.3.5 Use Case 5: Collaborative Writing

Description: Multiple users can collaborate on the same project with role-based access and real-time editing.

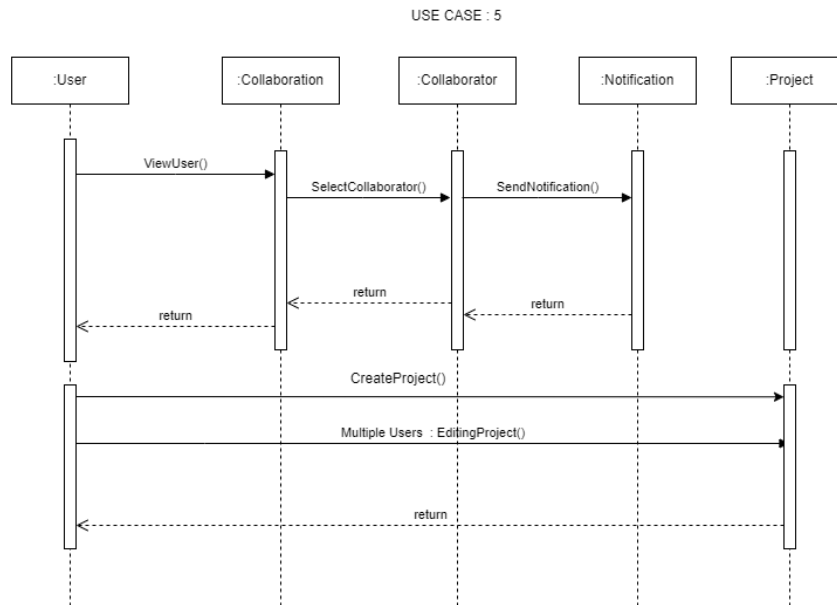


Figure 3.8: Sequence Diagram for Collaborative Writing

3.2.3.6 Use Case 6: View and Comment on Published Projects

Description: Readers can view published projects and leave comments or feedback.

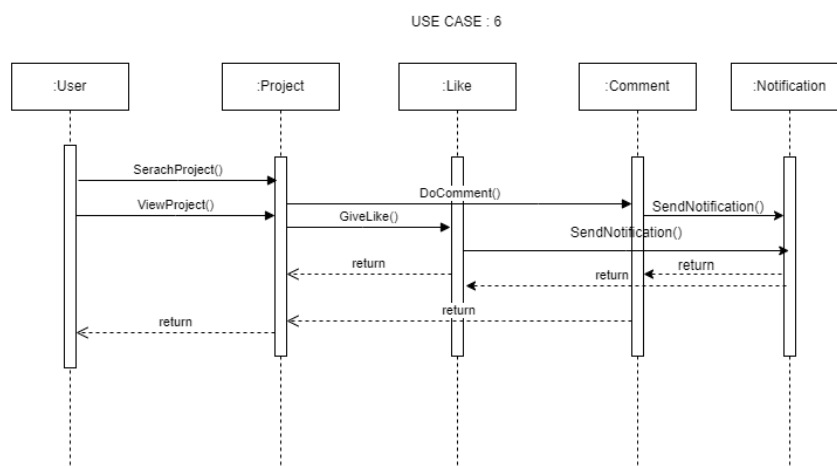


Figure 3.9: Sequence Diagram for View and Comment on Published Projects

3.2.3.7 Use Case 7: Manage Account Settings

Description: Users can update their profile, change email or password, and customize their preferences.

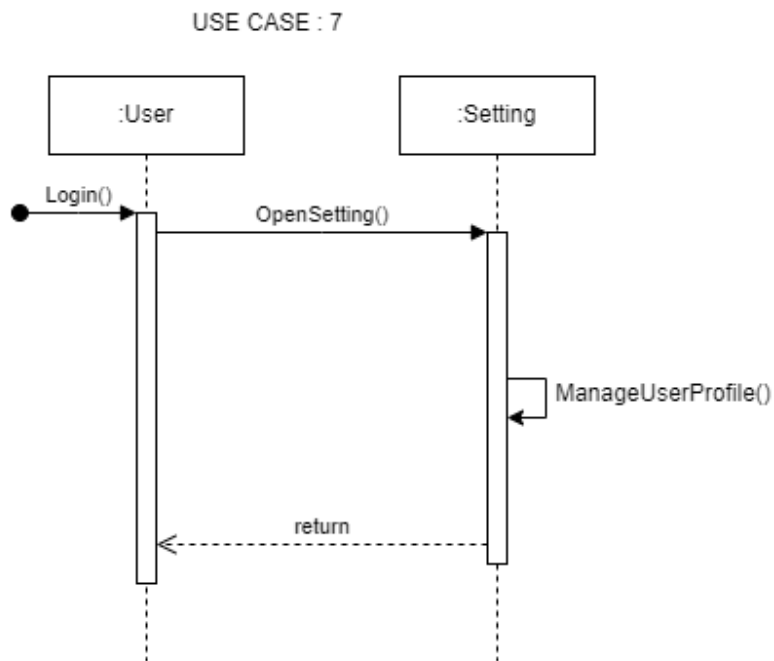


Figure 3.10: Sequence Diagram for Manage Account Settings

3.2.3.8 Use Case 8: AI-Assisted Grammar Check

Description: Writers can run grammar and spell checks on their projects using an AI-powered assistant.

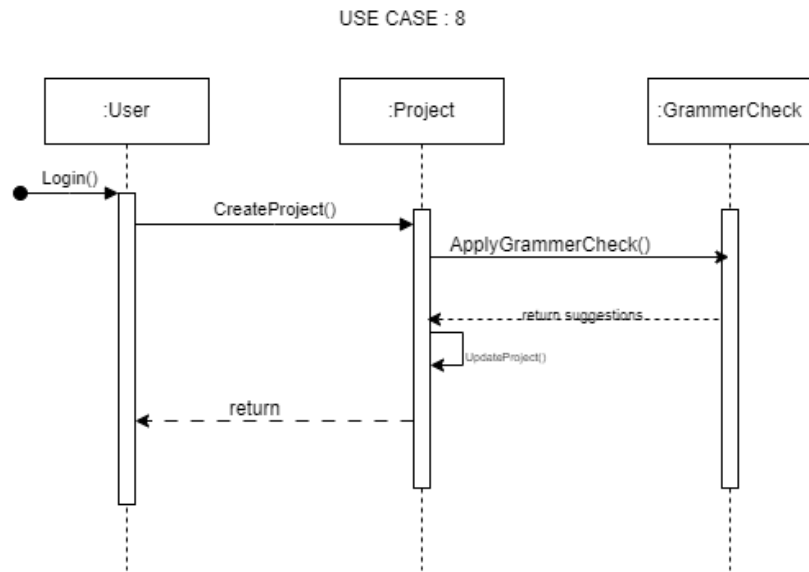


Figure 3.11: Sequence Diagram for AI-Assisted Grammar Check

3.2.3.9 Use Case 9: Manage Characters

Description: Writers can create, edit, and track character profiles within a project.

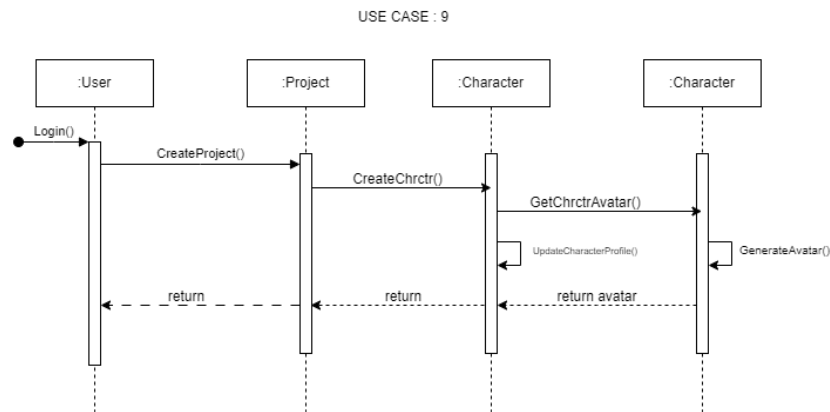


Figure 3.12: Sequence Diagram for Manage Characters

3.2.3.10 Use Case 10: Switch Language (English/Urdu)

Description: Users can toggle between English and Urdu for both the interface and content creation.

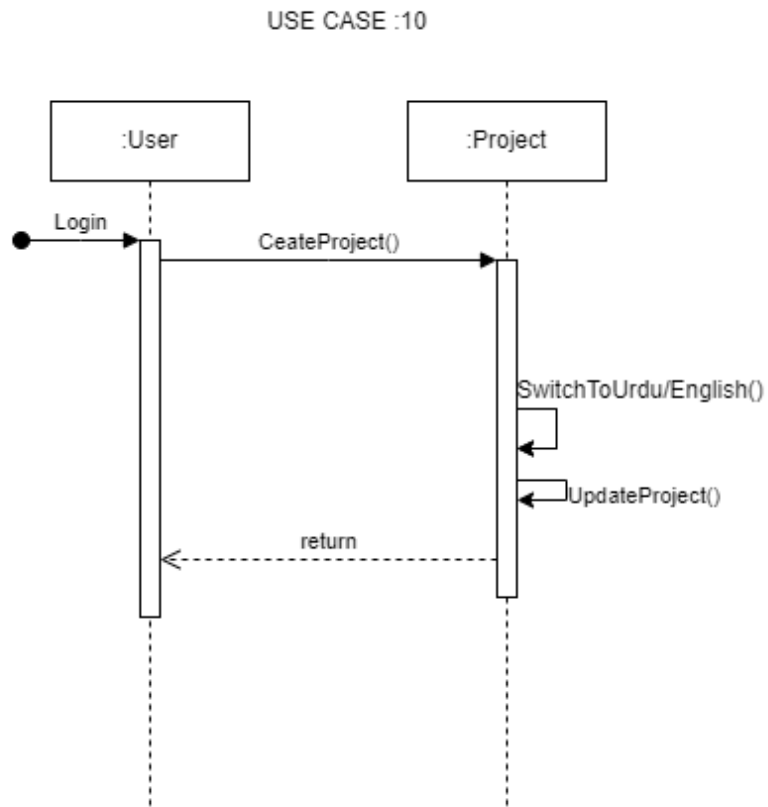


Figure 3.13: Sequence Diagram for Switch Language (English/Urdu)

3.2.3.11 Use Case 11: Customize Workspace

Description: Writers can change the appearance of their workspace and adjust settings to optimize their writing environment.

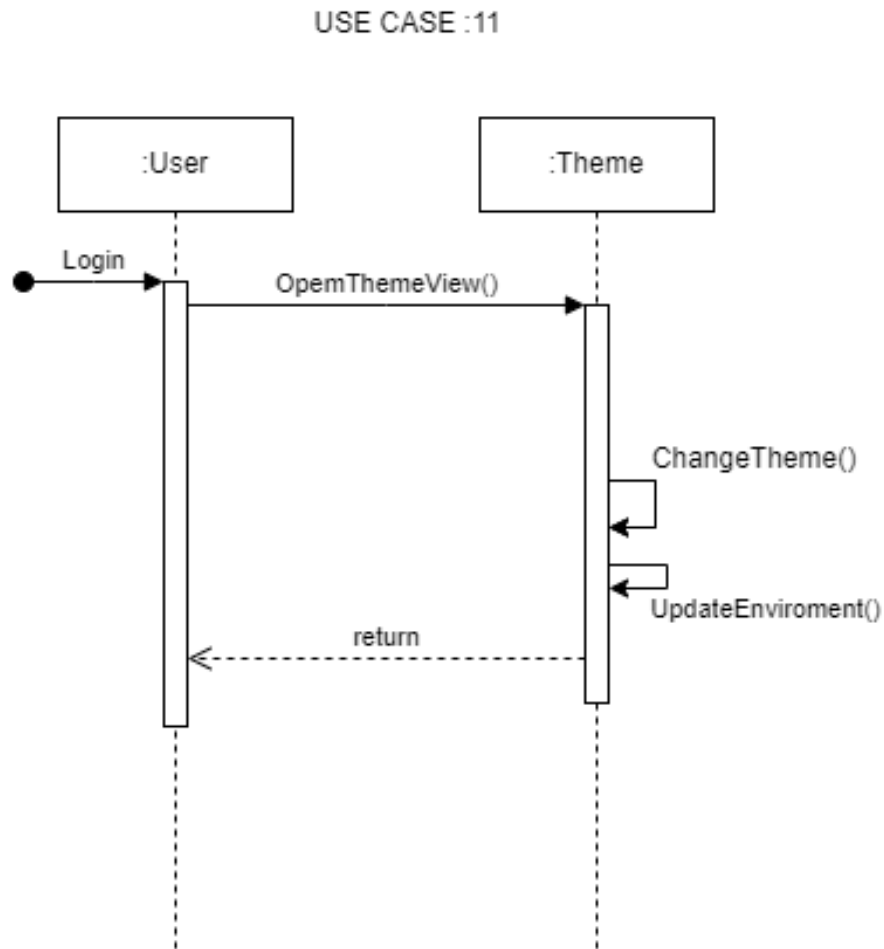


Figure 3.14: Sequence Diagram for Customize Workspace

3.2.3.12 Use Case 12: Export Project (PDF, DOCX, etc.)

Description: Allows users to export their entire project in various formats such as PDF or DOCX.

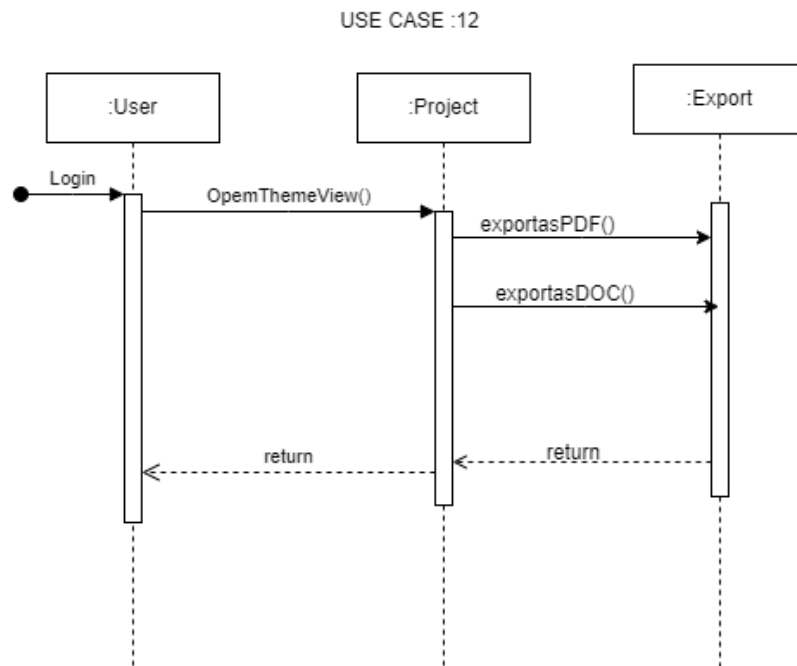


Figure 3.15: Sequence Diagram for Export Project

3.2.3.13 Use Case 13: Version Control

Description: Users can track changes, view the history of edits, and restore previous versions of their project.

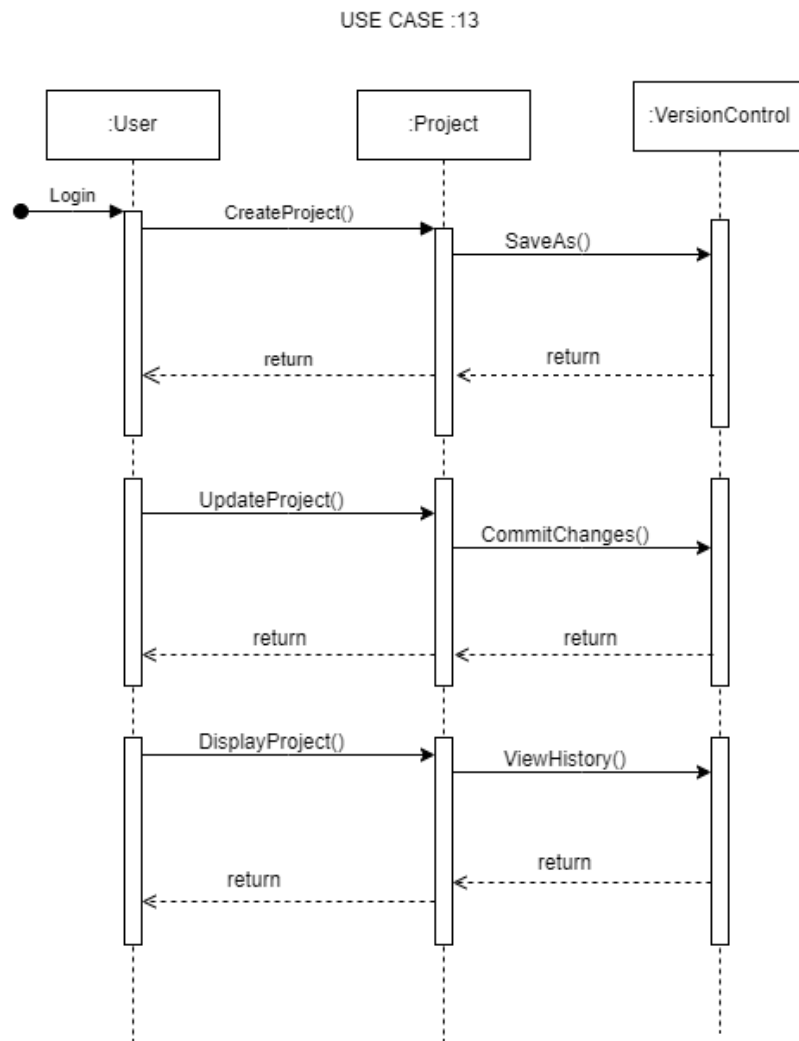


Figure 3.16: Sequence Diagram for Version Control

3.2.3.14 Use Case 14: AI-Assisted Writing Assistant (ChatBot)

Description: Users can interact with an AI chatbot to get suggestions, guidance, or answers to questions during the writing process.

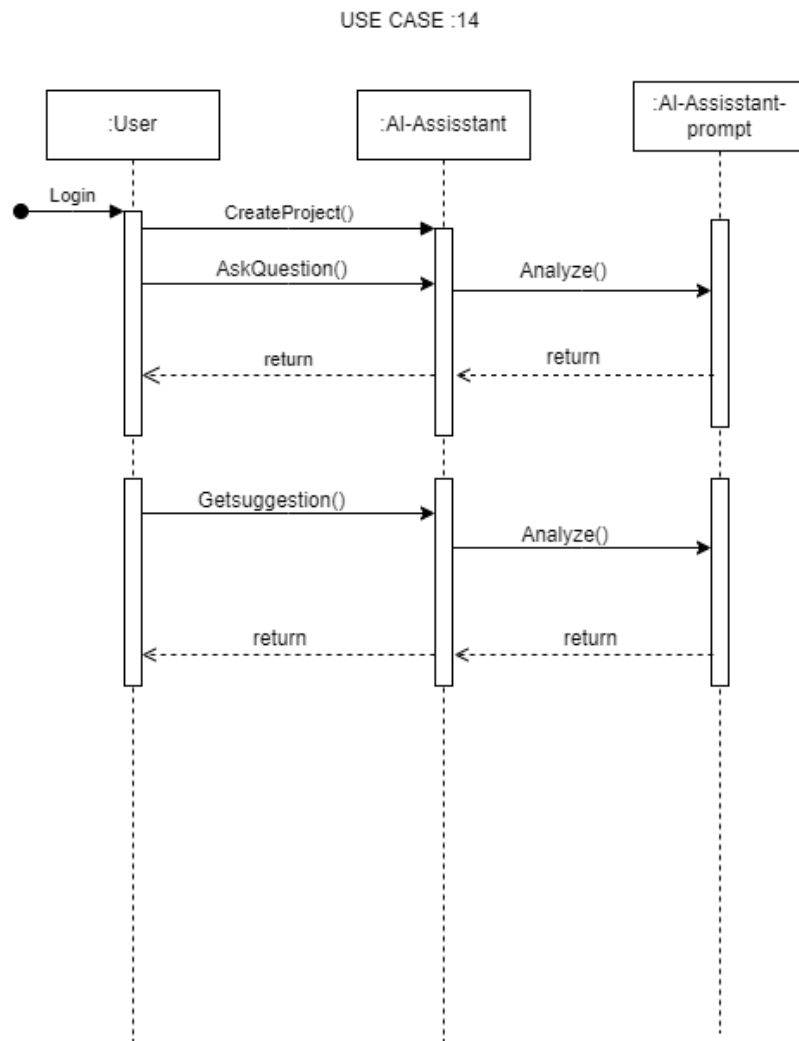


Figure 3.17: Sequence Diagram for AI-Assisted Writing Assistant (ChatBot)

3.2.3.15 Use Case 15: Manage Project (Edit, Delete)

Description: Writers can edit the details of their project or delete it if necessary.

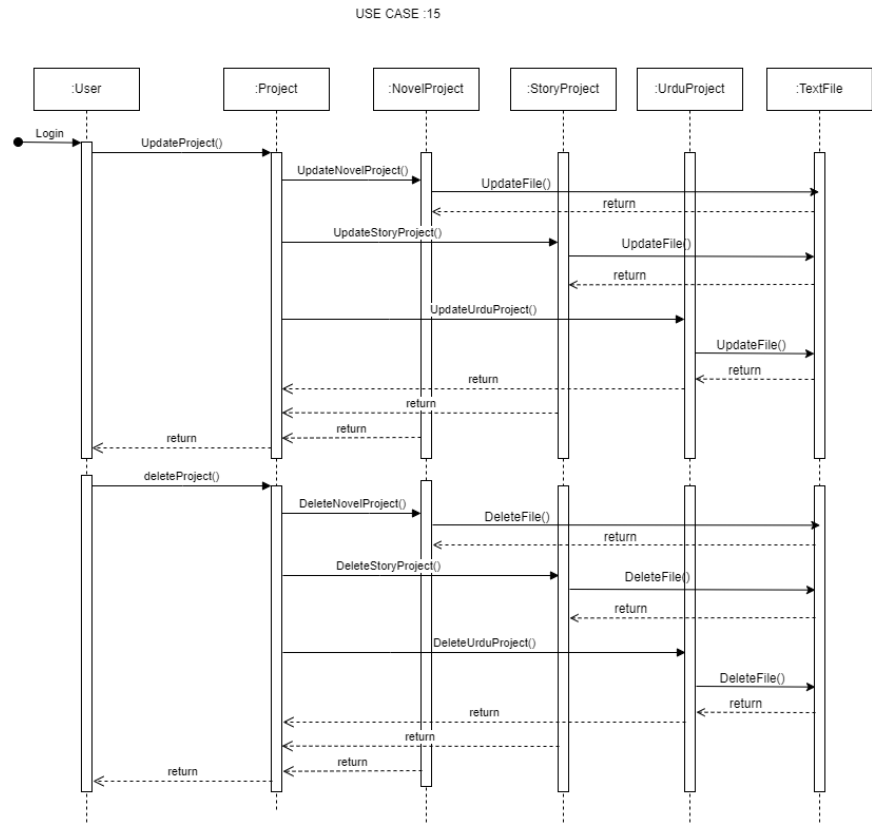


Figure 3.18: Sequence Diagram for Manage Project

3.2.3.16 Use Case 16: View Analytics and Writing Metrics

Description: Provides writers with analytics on word count, session duration, chapter progression, and more.

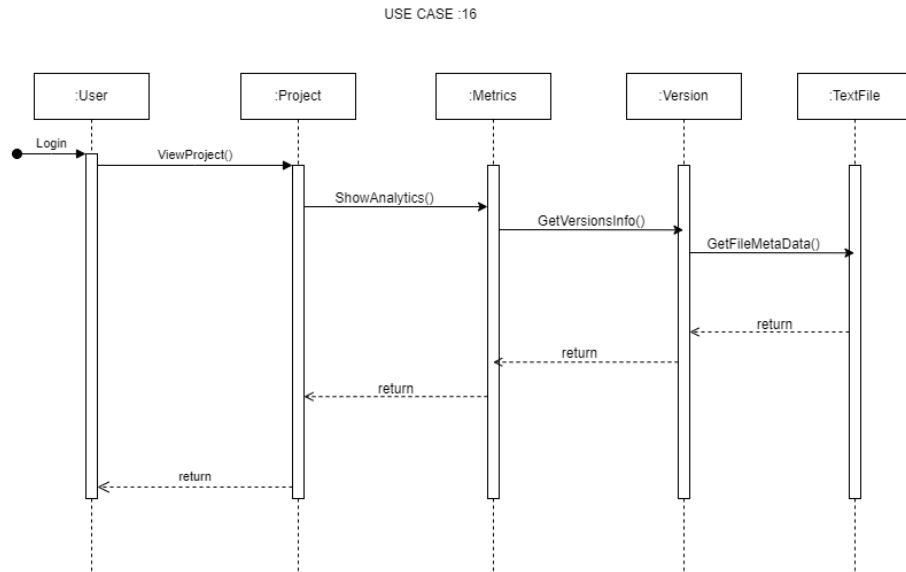


Figure 3.19: Sequence Diagram for View Analytics and Writing Metrics

3.2.3.17 Use Case 17: Publish Completed Project

Description: Allows writers to share their completed work with readers, publishers, or others so others can view it, like, or comment on it.

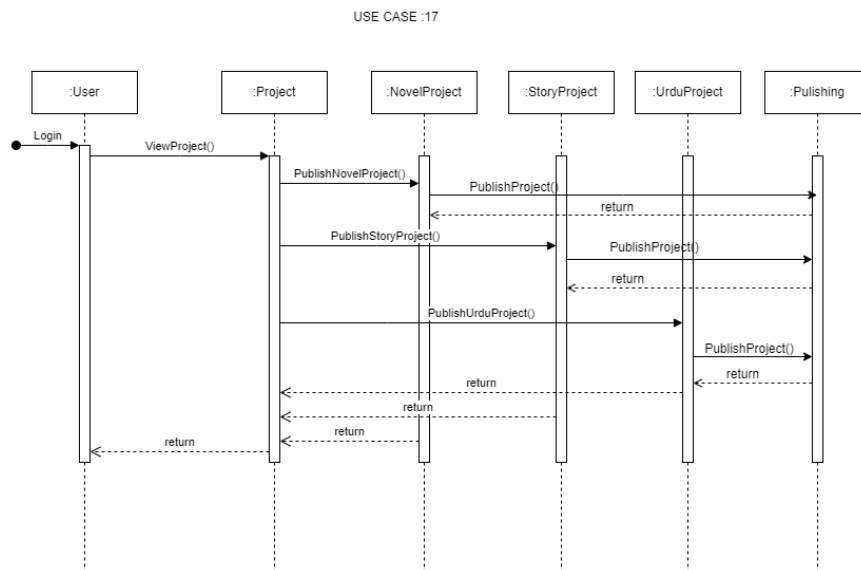


Figure 3.20: Sequence Diagram for Publish Completed Project

3.2.3.18 Use Case 18: Backup and Restore

Description: Users can create a backup of their project data and restore it in case of data loss.

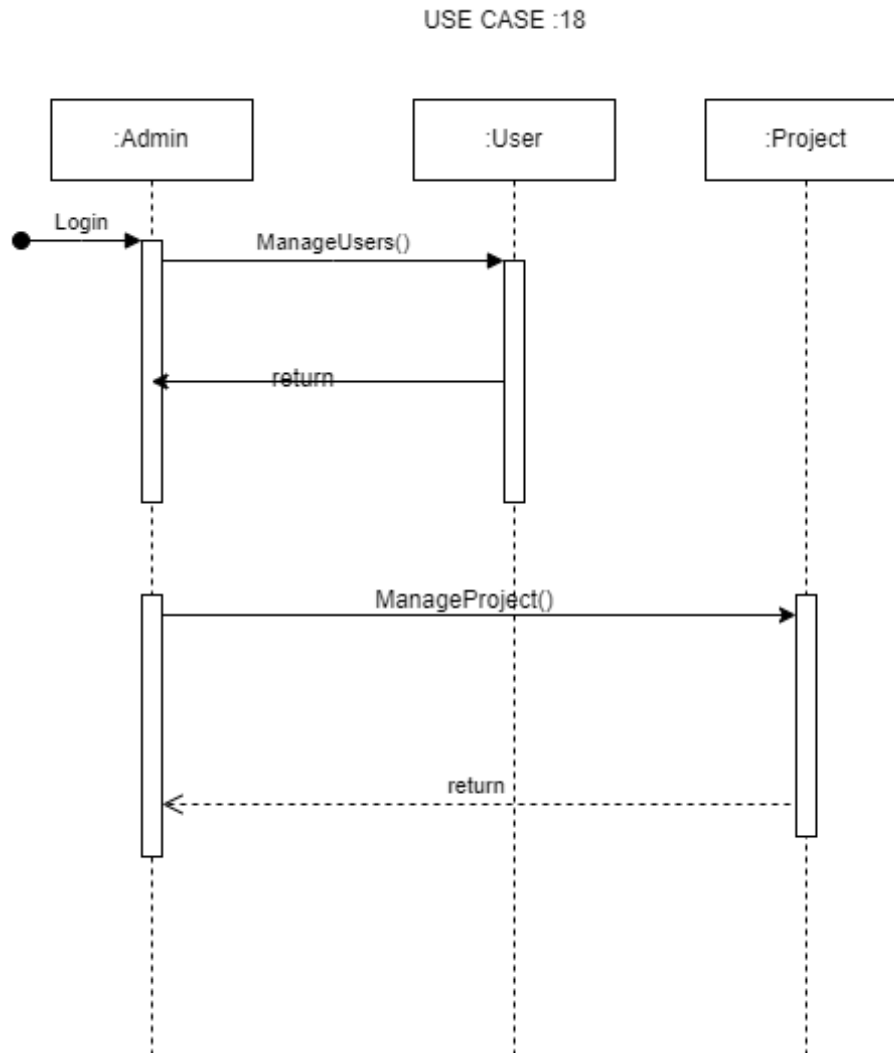


Figure 3.21: Sequence Diagram for Backup and Restore

3.3 Data Design

3.3.1 Data Transformation and Structures

VerseCraft transforms the creative writing domain into structured data entities, ensuring efficient storage, retrieval, and processing. The major system entities include **USER**, **Project**, **Chapter**, **Notes**, **Notification**, **Version**, **Admin**, **AI-Prompt**, **TextFile**, **ActivityLogs**, **Feedback**, **Theme**, **Collaborator**, and **Publishing**.

3.3.2 Data Entities and Relationships

1. USER:

- **Attributes:** USERID (Primary Key), Email, Profile, Age, Gender, Password, Description
- **Relationships:**
 - One-to-Many with **Project** (a user can have multiple projects)
 - One-to-Many with **ActivityLogs** (a user can have multiple activity logs)
 - One-to-Many with **Feedback** (a user can provide multiple feedback entries)
 - Many-to-Many with **Project** through **Collaborator** (users can collaborate on multiple projects)
 - One-to-Many with **Notification** (a user can receive multiple notifications)
 - One-to-Many with **AI-Prompt** (a user can have multiple AI interactions)

2. Project:

- **Attributes:** PROJECTID (Primary Key), Description, AuthorID (Foreign Key to USER), CreatedDate, Type, Genre, Tags, Title
- **Relationships:**
 - One-to-Many with **Chapter** (a project can have multiple chapters)
 - One-to-Many with **Notes** (a project can have multiple notes)
 - One-to-Many with **Version** (a project can have multiple versions)
 - One-to-One with **Publishing** (a project has one publishing status)
 - Many-to-Many with **USER** through **Collaborator**
 - One-to-Many with **Feedback** (a project can have multiple feedback entries)
 - One-to-Many with **AI-Prompt** (AI interactions related to the project)

3. Chapter:

- **Attributes:** CHAPTERID (Primary Key), PROJECTID (Foreign Key), Description, AuthorID (Foreign Key to USER), CreatedDate, Type
- **Relationships:**
 - Belongs to a **Project**
 - One-to-Many with **Notes** (a chapter can have multiple notes)
 - One-to-Many with **Feedback** (a chapter can have multiple feedback entries)

4. Notes:

- **Attributes:** NOTEID (Primary Key), PROJECTID (Foreign Key), Description, AuthorID (Foreign Key to USER), CreatedDate, Type
- **Relationships:**
 - Belongs to a **Project**
 - May be associated with a **Chapter**

5. Notification:

- **Attributes:** NotificationID (Primary Key), Description, SenderID (Foreign Key to USER), ReceiverID (Foreign Key to USER), CreatedDate, Type
- **Relationships:**
 - Sent from a **USER** to a **USER**

6. Version:

- **Attributes:** VersionID (Primary Key), Description, AuthorID (Foreign Key to USER), CreatedDate, PROJECTID (Foreign Key), ProjectType
- **Relationships:**
 - Belongs to a **Project**

7. Admin:

- **Attributes:** AdminID (Primary Key), Name, Email, Password
- **Relationships:**
 - Manages **USER** accounts and **Project** content
 - Can send **Notifications**

8. AI-Prompt:

- **Attributes:** PromptID (Primary Key), Response, Question, CreatedDate, Type, USERID (Foreign Key)
- **Relationships:**
 - Linked to a **USER**
 - May be associated with a **Project** or **Chapter**

9. TextFile:

- **Attributes:** TextFileID (Primary Key), Content, AuthorID (Foreign Key to USER), CreatedDate, Type

- **Relationships:**
 - May be associated with a **Project** or **Chapter**

10. ActivityLogs:

- **Attributes:** LogID (Primary Key), DateTime, UserID (Foreign Key), ActionPerformed
- **Relationships:**
 - Belongs to a **USER**

11. Feedback:

- **Attributes:** FeedbackID (Primary Key), Likes, Comments, CreatedDate, UserID (Foreign Key), PROJECTID (Foreign Key)
- **Relationships:**
 - Provided by a **USER** on a **Project** or **Chapter**

12. Theme:

- **Attributes:** ThemeID (Primary Key), Description, ThemeName, ColorScheme, Type
- **Relationships:**
 - Can be applied by a **USER** to customize the workspace

13. Collaborator:

- **Attributes:** CollaborationID (Primary Key), PROJECTID (Foreign Key), UserID (Foreign Key), Role, CreatedDate
- **Relationships:**
 - Links **USERS** and **Projects** for collaborative access

14. Publishing:

- **Attributes:** PublishingID (Primary Key), PROJECTID (Foreign Key), PublishingDate, AuthorID (Foreign Key to USER), Status, Type
- **Relationships:**
 - Associated with a **Project**

3.3.3 Database Schema

MongoDB Collections:

- users
- projects
- chapters
- notes
- notifications
- versions
- admins
- ai_prompts
- text_files
- activity_logs
- feedbacks
- themes
- collaborators
- publishings

3.3.4 Storage and Processing

- **User Data:** Stored in the users collection with secure password hashing and encryption for sensitive information.
- **Project Data:** Each project document contains references to its chapters, notes, versions, and collaborators, enabling efficient retrieval and management.
- **AI Data:** AI interactions are stored in the ai_prompts collection, linked to their respective users and projects.
- **Collaboration Data:** Managed through the collaborators collection, facilitating role-based access and real-time synchronization.

- **Activity Logs:** User activities are recorded in the `activity_logs` collection for auditing and monitoring purposes.
- **Feedback Data:** Stored in the `feedbacks` collection, linked to projects or chapters, allowing users to view and respond to feedback.

3.3.5 Data Flow

1. **User Interaction:** Users interact with the frontend to create or manage projects, chapters, notes, and other entities.
2. **Backend Processing:** The backend handles requests, interacts with the database, and communicates with the AI layer as needed.
3. **AI Integration:** AI tools analyze content and provide suggestions, which are then stored and displayed to the user.
4. **Data Storage:** All entities are persistently stored in MongoDB, ensuring data integrity and availability.

3.3.6 Security Measures

- **Authentication:** Implement this using secure tokens for better management of user sessions.
- **Authorization:** Role-based access control is to make sure writers/readers can only access and update authorized projects and data.
- **Data Encryption:** Data is encrypted both in transit and at rest.
- **Regular Backups:** Automatically save the database to prevent loss.
- **Audit Logging:** All user activities are logged for security monitoring and compliance.

3.3.7 Scalability Considerations

- **Database Sharding:** MongoDB's sharding capabilities allow for horizontal scaling as the user base and data volume grow.
- **Load Balancing:** Distributing incoming requests across multiple server instances ensures optimal performance and availability.

- **Microservices Architecture:** Potential future migration to microservices for individual modules (e.g., AI services) to enhance scalability and maintainability.

3.3.8 Database Schema Diagram

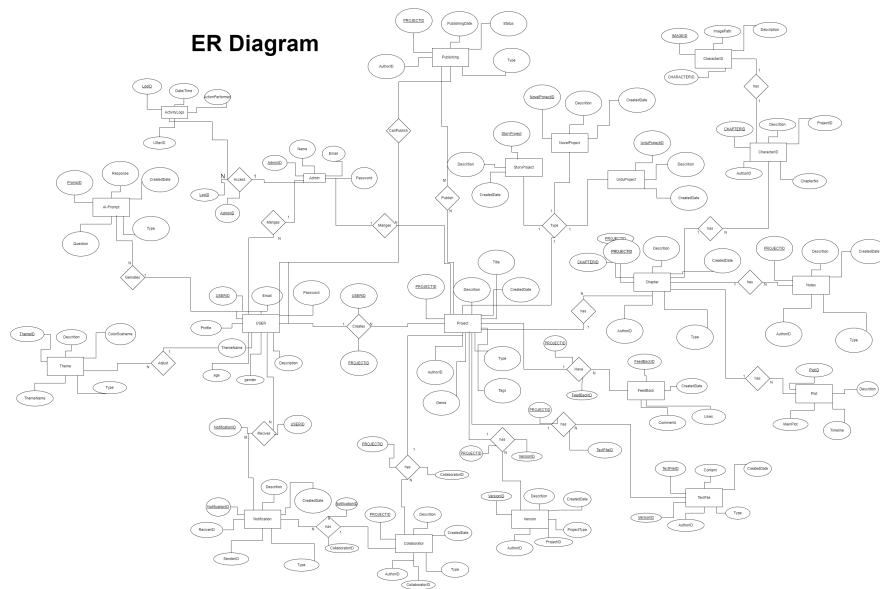


Figure 3.22: Database Schema Diagram for VerseCraft

3.3.9 Summary

The architectural design of VerseCraft leverages a comprehensive data model that encompasses all aspects of the writing workflow, from user management to project collaboration and AI integration. The data entities and relationships are meticulously structured to support the functional requirements outlined in the system's use cases. The use of MongoDB collections facilitates flexible and scalable data storage, while robust security measures ensure data integrity and user privacy. Scalability considerations like database sharding and load balancing prepare the system for growth, ensuring a reliable and responsive experience for users. This structured approach lays a solid foundation for developing a feature-rich, reliable, and user-friendly writing workflow management tool.

Chapter 4

Conclusions

This report details the design and implementation of VerseCraft, a comprehensive writing workflow management tool. VerseCraft addresses common writing challenges such as disorganization, writer's block, and maintaining grammatical accuracy through structured modules supporting project organization, collaborative writing, and automated assistance. The platform's modular architecture ensures scalability and maintainability, providing a robust and flexible foundation for managing writing projects. Detailed use case analysis, sequence diagrams, and a well-defined database schema guided the development process.

VerseCraft integrates services such as grammar checking and writing suggestions, designed to enhance the writing process and empower writers with the tools they need to create and manage their work effectively. The platform offers a centralized and organized approach to writing, aiming to foster a more productive and enjoyable creative experience.

Bibliography

- [1] Lei Huang, Jiaming Guo, Guanhua He, Xishan Zhang, Rui Zhang, Shaohui Peng, Shaoli Liu, and Tianshi Chen. Ex3: Automatic novel writing by extracting, excelsior and expanding. *arXiv preprint*, 2024.
- [2] Xuezhe Ma, Xiaomeng Yang, Wenhan Xiong, Beidi Chen, Lili Yu, Hao Zhang, Jonathan May, Luke Zettlemoyer, Omer Levy, and Chunting Zhou. Megalodon: Efficient llm pretraining and inference with unlimited context length. *arXiv preprint*, 2024.
- [3] Sonia Meyer, Shreya Singh, Bertha Tam, Christopher Ton, and Angel Ren. A comparison of llm finetuning methods & evaluation metrics with travel chatbot use case. *arXiv preprint*, 2024.